

Tic-Tac-Toe 분석 프로젝트

1. Introduction

이번 프로젝트는 틱택토(Tic-Tac-Toe)라는 간단한 게임을 기반으로, 기존의 게임과 달리 제한된 정보와 난수의 환경을 구성해서 완벽한 플레이를 지양하고 있습니다.

플레이어는 minimax 알고리즘을 활용한 두 AI (O,X)이며, 이들은 상대가 최선을 다해 플레이 하는 것을 계산하면서 최적의 움직임을 탐색하려는 특성을 지니고 있습니다.

2. How It works

- 사용 환경

사용 환경	활용 언어	분석 패키지	게임 종목	알고리즘
Visual Script Code	Python	pandas matplotlib.pyplot	TIC-TAC-TOE	minimax

- 게임 환경 :

Tic-Tac-Toe (3×3 기본 격자판)

- AI 행동 패턴 :

- 선/후공 결정 :

매 게임마다 X / O 중 랜덤으로 선공 순서가 결정됩니다.

```
def play_game():
    game = TicTacToe()
    letter = random.choice(["X", "O"])
```

- AI는 minimax 알고리즘을 사용하여 매 턴마다 10%확률로 완전히 무작위로 수를 둔 후 가능한 모든 수를 예상하며 최선의 결과를 선택하는데, 이는 최선의 수를 계산하는 AI가 약간의 오류를 가지게 하여 현실적인 불완전성을 반영하기 위함입니다.
- "Depth"는 AI가 몇 수를 미리 계산할지 결정하는 매개변수로, 1~4 사이로 무작위 선택하여 매 게임마다 예측 깊이를 다르게 하여 다양한 변수를 만들고자 했습니다

```
while game.empty_squares():
    if random.random() < 0.1:
        move = random.choice(game.available_moves())
    else:
        random_depth = random.randint(1, 4)
        move = minimax(game, letter, depth=random_depth)['position']
```

- 시뮬레이션 실행 :

- 한 세트는 50판의 게임으로, 총 5세트를 진행하였습니다.

```
if __name__ == "__main__":
    n_sets = 5
    n_games_per_set = 50
```

- 데이터 기록 :

- 각 세트마다 O, X의 승리 수와 무승부 수를 기록하여 요약과 함께 터미널에 출력한 후, matplotlib.pyplot을 활용해 막대 그래프로 수치를 보여줍니다.

```
all_results = []

for i in range(n_sets):
    set_result = simulate_games(n_games_per_set)
    print(f"Set {i+1}: {set_result}")
    all_results.append(set_result)

df = pd.DataFrame(all_results, index=[f"Set {i+1}" for i in range(n_sets)])
print("\nn=== Summary ===")
print(df)
```

```
df.plot(kind="bar", figsize=(10,6))
plt.title("Tic-Tac-Toe AI Simulation Results (5 Sets)")
plt.xlabel("Set")
plt.ylabel("Number of Wins / Draws")
plt.xticks(rotation=0)
plt.legend(title="Result")
plt.tight_layout()
plt.show()
```

3. Analysis

3-1. 상세 분석

프로그램을 구동한 결과, 5세트 동안 O의 평균 승리 수는 24, X의 평균 승리 수는 22, 무승부는 평균 4회로 나타났습니다. 이는 난수로 인한 불완전성에도 불구하고, 승패 간의 편향이 크게 발생하지 않았음을 알 수 있습니다.

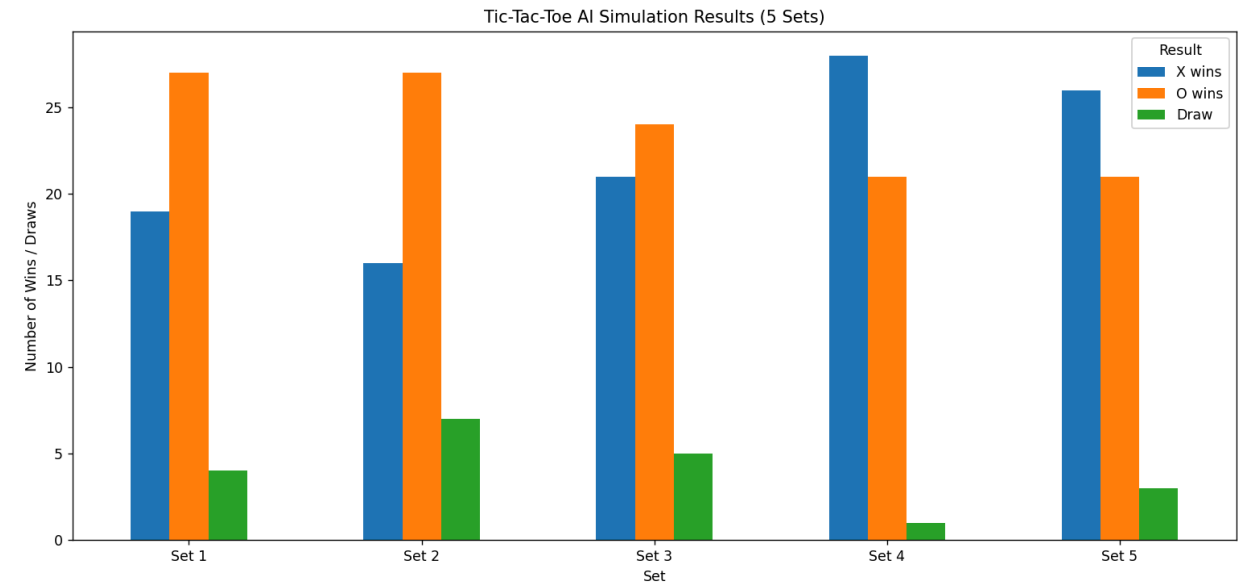
```
Set 1: {'X wins': 19, 'O wins': 27, 'Draw': 4}
Set 2: {'X wins': 16, 'O wins': 27, 'Draw': 7}
Set 3: {'X wins': 21, 'O wins': 24, 'Draw': 5}
Set 4: {'X wins': 28, 'O wins': 21, 'Draw': 1}
Set 5: {'X wins': 26, 'O wins': 21, 'Draw': 3}

=== Summary ===
      X wins  O wins  Draw
Set 1      19     27     4
Set 2      16     27     7
Set 3      21     24     5
Set 4      28     21     1
Set 5      26     21     3
PS D:\GitHub\tic-tac-toe-ai-battle>
```

[Image 1] 각 세트의 승/패/무효 결과 요약 출력

[Table 1] Set별 O,X 승리 및 무승부 결과 요약

Set	X wins	O wins	Draws
1	19	27	4
2	16	27	7
3	21	24	5
4	28	21	1
5	26	21	3



[Image 2] 5세트 결과를 matplotlib로 시각화한 그래프

이와 달리, 기존의 코드에 난수를 부여하지 않은 상태에서 두 AI가 최선의 수를 계산하도록 설정했을 땐 50판 기준으로 전부 무승부임을 확인할 수 있었습니다.

```
Simulated 50 games:
{'X wins': 0, 'O wins': 0, 'Draw': 50}
PS D:\GitHub\tic-tac-toe-ai-battle>
```

[Image 3] 난수를 제거한 경우의 결과 (모두 무승부)

4. Conclusion

앞선 결과에서 볼 수 있듯, 저는 단순한 구조의 틱택토 게임에 난수를 삽입함으로써 일부러 불완전한 환경을 조성하고, 그 안에서 AI가 어떻게 최적의 수를 선택하려 하는지를 관찰하고자 했습니다.

처음 이 프로젝트를 구상할 때는, 무엇이든 뚫는 창과 무엇이든 막는 방패처럼 완벽한 AI들끼리의 대결 구도에서 일부러 불완전성을 주었을 때 어떤 변화가 생기는지 확인해보고 싶었습니다.

결과적으로는 난수의 삽입 여부에 따라 승패 분포에 유의미한 차이가 나타남을 알 수 있었습니다.

이러한 경험을 통해, 단순한 구조에 조금만 변수를 주고 시야를 달리하면 전혀 다른 결과가 나올 수도 있음과 함께, AI와 보안환경에서도 예측 불가능한 변수들이 언제나 존재할 수 있음을 체감하는 계기가 되었습니다.

추가 경험: webshell-upload-vuln 프로젝트

팀 프로젝트 이후에 시작했던 제 이전 프로젝트도 별도로 백업 환경을 구축해, 취약점 간 연계에 따른 추가적인 위협 요소를 탐색하면서 보안을 바라보는 제 시야도 함께 넓힐 수 있었습니다.

 자세한 건 다음의 링크에서 확인할 수 있습니다 :

<https://github.com/BarnOwl5/webshell-upload-vuln>

작은 실험이었지만, 이번 경험을 통해 얻은 관점의 변화가 앞으로도 보안에 대한 유연한 생각으로 이어질 수 있도록 노력하고 싶습니다.

끝까지 읽어주셔서 감사합니다!